

Prediction of HER2-binding nanobodies from frozen protein language model embeddings

1. Verification of the dataset's stated properties

Every stated property holds except the labelling threshold, which is misstated. Claims were re-derived rather than assumed (`src/nanobody/data.py::verify_splits`, asserted in `tests/test_pooling.py`). Confirmed: the sign convention (94.3% of binder cells negative in the five plain-HER2 columns, median -1.77 , against 0.0% and $+1.65$ for non-binders); the hole in the middle (no labelled row with $|\text{target}| < 1.3195$); cluster-disjoint splits (0 shared clusters or sequences train-val-test, 0 duplicates); class balance (38.3% / 36.4%); the ± 500 markers (9 train, 1 val); and the cluster structure that makes ungrouped CV read high (8,763 clusters, 83.4% singletons, but the other 16.6% hold 46.1% of rows with 97.5% constant labels).

One claim is wrong as stated. The README says `binary_label` is "essentially `target_IE_score ≤ -1.32`", which agrees on only 98.9% of rows. The exact rule is `binary_label = 1 ⇔ target_IE_score < 0` — binders span $[-500, -1.3195]$, non-binders $[+1.3230, +500]$, nothing between.

This discrepancy is minor in magnitude but consequential in interpretation. It establishes that the two targets are one variable observed at two resolutions, with the decision boundary at exactly zero. Task 2's "oracle knowing only the class" is therefore a sign predictor, and any Task 2 model exceeding $\rho \approx 0.835$ must be ordering observations *within* a class.

2. Feature admissibility and excluded columns

`test_public.csv` ships `id` and `sequence` only, so the operative test for every other column is: **could this value exist for a nanobody that has never been screened?** Full table in `src/nanobody/data.py::column_verdicts()`.

Mechanical exclusions. The 26 `Marin-*` panning columns are per-experiment enrichment from the screen that produced the label — `target_IE_score` is an aggregation of the 15 HER2 ones (Spearman 0.92 against their row mean), so they are not leakage-adjacent, they are the answer. `background_IE_score` is the sibling aggregation over the 11 controls (Spearman 0.83). `label` and `binary_label` are targets; feeding `binary_label` to a Task 2 model confers $\rho \approx 0.835$ without any learned contribution.

The four genuinely arguable ones:

- **id** — **excluded**. Assigned in descending order of total read count, so the integer in `ONU<n>` is a monotone function of library abundance: an assay measurement wearing an identifier's clothes.
- **total** — **excluded**. Read count summed across all panning experiments, HER2 ones included. Abundance in the *output* of a screen is a consequence of binding.
- **#Cluster** — **excluded**. A cluster index into the full library ordering (max 472,250 against 8,763 clusters present here), so derived from library-wide abundance like `id`.
- **Cluster_size** — **excluded, and this is the interesting one**. It is *not* the number of rows the cluster contributes to the file: they match for only 1.4% of rows (median `Cluster_size` 53, median rows-per-cluster 1). So it counts members in the **full unlabelled library**, which makes it a property of the alpaca's repertoire rather than of the HER2 panning — and arguably computable for any sequence you can map back to that library. I still exclude it, because (a) it cannot be computed for a genuinely novel design, which is the use case the exercise is about, and (b) clonal expansion in an immunized animal is itself partly a response to the immunogen, so it correlates with binding through a path a fresh candidate does not have. I would report it as a covariate, never ship it as a feature.

Cluster is used for CV grouping only, never as a feature.

3. Task 1 — binder classification

3.1 Representation: sequence to model input

Frozen `biohub/ESMC-300M`, one forward pass, **masked mean pooling over real residues only**. The tokenizer adds BOS/EOS and pads each batch to its longest member, so pooling the raw tensor averages in padding and makes a sequence's vector depend on its batch-mates — it still runs, the numbers are just worse and not reproducible across batch sizes. The mask is `attention_mask & ~special_tokens_mask`.

Batch-invariance check. `embed_esm.py --check` embeds one sequence alone and again inside a batch of 32 mixed lengths and requires the pooled vectors to agree. The same property is unit-tested on synthetic tensors in `tests/test_pooling.py`, including a guard that the suite fails if pooling is replaced by a naive `.mean(dim=1)`.

An independent confirmation from the cache itself: the stored residue counts span 100–148, exactly the raw sequence-length range. Had BOS/EOS leaked into the mask they would read 102–150.

3.2 Layer choice

Five layers cached in one pass (6/12/18/24/30) and ranked by logistic-regression val AUROC, because that measures how *linearly separable* the representation is, which is the property a small head can exploit.

Pooled layer	val AUROC (logreg)
6	0.9173
12	0.9339
18	0.9254
24	0.9356
30 (last)	0.9237

The last layer is not the best. Layer 24 beats layer 30 by 0.0119, and even layer 12 beats it. This is the expected behaviour for a masked-LM — the final layers specialise for token reconstruction rather than sequence-level semantics — but it is measured here, not assumed. Everything downstream uses **layer 24**. The trend is not monotone (18 dips below 12), which I would not read too much into at this spread.

3.3 Architecture and training

`Linear(960 → 256) → ReLU → Dropout(0.3) → Linear(256 → 1)`, `BCEWithLogitsLoss`, AdamW (lr 1e-3, weight decay 1e-4), batch 128, ≤60 epochs, early stopping on val AUROC (patience 10), seeds 0/1/2 averaged. Inputs standardised on train statistics. No class reweighting — 38.3% vs 36.4% positives does not need it.

3.4 Validation results

Model	val AUROC	AUPRC	Accuracy	F1	MCC
majority class	0.5000	—	—	—	—
1-mer composition + logreg	0.7209				
1-NN 3-mer retrieval	0.9140				
3-mer TF-IDF + logreg	0.9430				
logreg on ESMC layer 24	0.9356	0.9077	0.8620	0.7973	0.6971
MLP on ESMC layer 24 (submitted)	0.9625	0.9526	0.9103	0.8654	0.8064

The four k-mer rows are measured on our val split (`uv run python baselines_kmer.py`). They sit below the test-set references quoted in `evaluate.py` (0.799 / 0.931 / 0.967) by 3–8 points. This suggests the validation split is somewhat harder than the withheld test split; accordingly, **margins rather than absolute values should be compared across the two**.

Margin that matters: MLP – logreg on identical embeddings = **+0.0269**, against a seed-to-seed spread of ±0.0020 (seeds 0/1/2: 0.9645, 0.9597, 0.9632). The margin is ~13× the spread, so this is not noise: **the classifier is**

contributing, and the brief's expectation that the two would land within 0.004 of each other does not hold here.

The mechanism appears to be limited linear separability rather than limited information. A linear probe on mean-pooled ESMC (0.9356) underperforms 3-mer TF-IDF logistic regression (0.9430) on the same split; introducing a single hidden layer reverses the ordering (0.9625 against 0.9430, +0.0195). This indicates the discriminative signal is present in the representation but not linearly accessible — the inverse of the typical frozen-features result, in which the representation is decisive and the classifier incidental.

Grouped CV. `GroupKFold(Cluster)`, 5 folds, logreg: **0.9162 ± 0.0124** (folds 0.9342, 0.9117, 0.9017, 0.9066, 0.9270). Below the 0.9356 val figure, as expected — each fold trains on 80% of the data, and fold-to-fold spread of ±0.012 is a better guide to how much of a margin is meaningful than the val number alone.

3.5 Why a frozen representation transfers to this task

The representation transfers because this library varies almost exclusively in the CDR loops. ESMC was not exposed to a HER2 binding assay; it was trained on sufficient natural protein sequence to encode which residues are tolerated at which positions in a VHH fold. Because all 19,367 sequences share a single scaffold and differ principally in three loops, most of the variance the model encodes corresponds to CDR variation. The pooled embedding therefore approximates a learned, context-aware representation of the loops, which are the determinants of binding.

What it should therefore be bad at: anything requiring the *target*. The model has no representation of HER2 whatsoever, so it cannot distinguish a HER2 binder from a clone that binds something else well. It is learning "looks like the binders in this library", which is a property of this panning experiment, not of HER2 — and that is also the reason it would transfer poorly to a different antigen.

4. Task 2 — binding strength

4.1 Treatment of the ±500 censored rows

Dropped from regression training and from the val metrics (9 train, 1 val). They are "off the scale" markers, not measurements: keeping them lets ten rows dominate any squared-error gradient, and clipping them to the observed extreme invents a number the assay never produced. The grader drops the test ones, so dropping them from val keeps our number comparable. They are **kept** for classification, where only the sign is used and the sign is trustworthy.

4.2 Target transform

Skewness of the raw target is −10.2 (min −301, median +1.65). Spearman cannot be changed by a monotone transform of the *prediction*, so any difference below is about which rows dominate the loss.

Ridge on ESMC layer 24, penalty chosen by `RidgeCV` over `logspace(−1, 4, 12)`:

Transform	val p	within-binders	within-non	MAE	R ²
raw	0.5803	+0.0613	+0.0283	4.477	+0.102
percentile rank	0.6072	+0.0558	+0.0659	4.645	−0.006
sign(y)·log1p y 	0.6113	+0.0616	+0.0492	4.161	+0.034
winsorised 1/99	0.5947	+0.0734	+0.0298	4.247	+0.098

Both compressing transforms exceed the raw target by approximately 0.03, consistent with the proposed mechanism: at skewness −10.2, a small number of extreme clones dominate the squared-error loss. `signed_log1p` and rank are within 0.004 of each other — tied, on this evidence.

A note on the penalty: at the default `Ridge(alpha=1.0)` these fits are ill-conditioned (scipy reports `rcond` ≈ 2×10^{-8}) because the 960 pooled dimensions are strongly collinear. The fit returns numbers but they are not numerically trustworthy, so the ESMC ridge uses `RidgeCV`. The k-mer references keep `alpha=1.0`, since the brief specifies that exact configuration and sparse TF-IDF is not ill-conditioned there.

Reference, measured on our val split with 3-mer TF-IDF ridge: raw $\rho = 0.6153$ (within $+0.0571 / +0.0038$), percentile-rank $\rho = 0.6262$ (within $+0.0407 / +0.0195$). The rank target beats raw by ~ 0.011 here, same direction as the brief's test-set figures ($0.626 \rightarrow 0.654$).

4.3 Overall ρ is dominated by class separation

On the validation split the 3-mer ridge model attains $\rho = 0.6153$ overall, with within-class ρ of $+0.057$ (binders) and $+0.004$ (non-binders) — values indistinguishable from zero. The brief's central claim was reproduced rather than assumed. The model has learned to classify, and the evidence does not indicate it has learned anything further. The class oracle attains $\rho = 0.8335$ against the brief's quoted 0.835, which suggests the two splits behave comparably on this statistic; this is a single point of agreement and does not establish equivalence in general.

The headline metric is therefore attributable almost entirely to classification, and exceeding $\rho \approx 0.835$ requires within-class ordering. The proportion is not estimated here; the supporting observation is that the ridge model's within-class ρ is indistinguishable from zero while its overall ρ is 0.6153.

4.4 Architecture and the combination problem

One trunk, three heads: class logit, plus one within-class rank head per class. Loss = BCE + within-class rank MSE + a within-class pairwise margin loss (pairs sampled inside a class only, which is what stops it re-learning the boundary). **Early stopping on mean within-class ρ , not overall ρ** — selecting on overall ρ just picks the best classifier.

The combination step is the principal failure mode, and it fails without raising an error. Under the direct formulation $\text{pred} = -W \cdot p + w \cdot r$, the class probability p continues to vary within a class; for $W \gg w$ that variation dominates the ordering the rank head has learned. The submitted combination instead predicts the **expected global percentile rank**, with $\pi = P(\text{binder})$:

$$\text{rank} = p \cdot (\pi \cdot r_{\text{binder}}) + (1-p) \cdot (\pi + (1-\pi) \cdot r_{\text{non}})$$

An expectation over the class posterior rather than an arbitrary weighting, monotone in each input.

Same trained model (seed 0), three combinations:

Combination	val ρ	within-binders	within-non
expected_rank (submitted)	0.6267	+0.1302	+0.1172
weighted ($-100 \cdot p + r$)	0.6653	+0.0861	+0.0876
hard class assignment	0.5646	+0.1536	+0.1285

These measurements confirm the predicted failure. **weighted** attains the highest overall ρ ($+0.039$ relative to **expected_rank**) while reducing within-class ρ by approximately one third, consistent with the $-100 \cdot p$ term dominating the within-class ordering. **hard** exhibits the converse: within-class ordering is preserved in full, at a cost of 0.062 in overall ρ , because a misclassified clone is assigned to the opposite extreme of the global ordering.

expected_rank is submitted because it is the only one that is competitive on both, and because it is an expectation over the class posterior rather than a tuned fudge factor. As a seed ensemble it also reaches $\rho = 0.6695$, above the **weighted** single-seed figure, so the trade-off is not even costly. For completeness, the **hard** seed ensemble gives $\rho = 0.5775$ with within-class $+0.1597 / +0.1483$ — the highest within-class numbers I obtained, at a headline ρ below both k-mer references. Given the brief's stated preference I would defend that submission too; I chose **expected_rank** because it clears the references on the primary metric *and* roughly triples within-class ρ .

4.5 Final results

Model	val ρ	within-binders	within-non	MAE	R ²
3-mer ridge, raw (reference)	0.6153	+0.0571	+0.0038	4.510	+0.119
3-mer ridge, rank (reference)	0.6262	+0.0407	+0.0195	4.626	−0.005
class oracle (reference)	0.8335	undefined	undefined	3.125	+0.113
ESMC ridge, signed_log1p	0.6113	+0.0616	+0.0492	4.161	+0.034
our <code>prob_map</code> baseline	0.6200	+0.0481	+0.0427	3.740	+0.082
two-stage, mean of 3 seeds	0.6518	+0.1146	+0.1249	4.593	−0.003
two-stage, seed ensemble (submitted)	0.6695	+0.1175	+0.1326	4.593	−0.003

Seed spread on the two-stage ρ is ± 0.0178 (0.6267, 0.6627, 0.6659), so the gap to the rank-ridge reference (0.6262) is real but the gap to 0.654 is roughly one spread — I would call the submitted model *ahead* of the raw-target reference and *level-to-ahead* of the rank-target one on overall ρ .

The number that matters is the within-class pair. Every reference and every baseline here sits at +0.02 to +0.07; the two-stage model reaches +0.118 / +0.133, about 2–3 $\times$, and the direction is consistent across both classes and all three seeds. That is a small amount of genuine within-class ordering where the k-mer models have essentially none. It is also still a *small* number in absolute terms — see §5.

Within-class ρ is undefined rather than 0.0 for the oracle: a prediction that is constant inside a class has no ordering to score. Reporting it as 0.0 would imply a measurement that was not made.

4.6 Validity of `target_IE_score` as an affinity proxy

Two reasons it is not:

1. **It measures recovery, not binding.** Enrichment on HER2-coated beads confounds affinity with everything else affecting survival through panning: expression level in the phage/yeast system, display efficiency, protease resistance, avidity from multivalent display, and PCR/NGS amplification bias. A clone that expresses twice as well enriches better at equal K_D .
2. **The scale is not affinity-like and is censored.** Real affinities span orders of magnitude in K_D ; this is a statistical enrichment score pooled over conditions, with a hole through the middle (non-significant clones deleted) and ± 500 censoring at the edges. A rank is meaningful; a difference is not, and a ratio certainly is not.

4.7 Rationale for Spearman ρ over R²

Because the decision the number stands in for is a ranking one — pick the five strongest of a hundred candidates — and because the target's scale is not trustworthy (§4.6). With skewness −10.2, R² is dominated by a handful of extreme clones: a model can improve R² substantially by predicting one −301 row better while getting the ordering of everything else worse. Spearman is invariant to any monotone distortion of the scale, which is exactly the part of this target we do not believe. Expect negative R² alongside positive ρ for the rank-trained models — the ordering is informative, the predictions are simply not on the target's scale.

5. Principal limitations and proposed next steps

The weakness: within-class ρ of 0.12 is real but small, and I cannot yet say how small. The class boundary is readily separable (0.9625 AUROC; 0.9430 from 3-mers alone). The within-class ordering carries the practical value, and the two-stage model recovers a measurable amount: +0.118 / +0.133, consistent across both classes and all three seeds, against +0.02 to +0.07 for every other model evaluated. However, $\rho = 0.12$ accounts for little variance, and the present evidence does not distinguish *insufficient signal* from *insufficient model*. Because `target_IE_score` is an enrichment proxy whose within-class variance is substantially assay noise (§4.6), an upper bound below 1.0 is expected. Absent an estimate of that bound, $\rho = 0.12$ cannot be interpreted: it may represent most of the accessible signal or a small fraction of it.

A second weakness I tested and was wrong about. I expected mean pooling over ~ 129 residues to dilute the CDR signal that determines binding, and predicted that pooling over CDR3 only would beat it. It does not. Measured on the same cache (layer 24, MLP 3 seeds):

Pooling	Task 1 val AUROC	Task 2 val ρ	within-binders	within-non
whole chain	0.9641 ± 0.0014	0.6620	+0.1258	+0.1136
CDR3 only	0.9581 ± 0.0021	0.6603	+0.0892	+0.0851
both concatenated	0.9671 ± 0.0012	0.6816	+0.0965	+0.1237

CDR3-only is **worse** on both tasks — notably on within-class ρ , where it loses about a third. So the framework regions are not inert padding that dilutes a signal; they carry information the model uses. That is plausible in hindsight: CDR1 and CDR2 also vary and also contact the antigen, and CDR3-only pooling discards them along with the framework. The argument I reasoned from — that CDR3-pooled vectors exhibit $4.2\times$ the between-sequence dispersion of whole-chain vectors ($\sigma = 2.22$ against 0.53) — measures *variability* rather than *discriminative utility*. The two are not equivalent, and dispersion is a proxy that can be mistaken for evidence.

Concatenating both helps modestly — +0.0030 AUROC at $\sim 2\sigma$, consistent across seeds — but on Task 2 it buys overall ρ while *losing* within-binders (0.0965 vs 0.1258). By the criterion this task says matters most, whole-chain pooling wins, and both submitted models use it.

With another week, in priority order:

1. **Bound the noise ceiling** (*highest value, no GPU*). The 15 HER2 columns are separate measurements of the same quantity, so their within-class disagreement estimates the assay's reliability and upper-bounds any achievable within-class ρ . Without it, $\rho = 0.12$ is uninterpretable. Those columns stay excluded as *features*; using them to characterise label noise is a different use, and I would say so explicitly.
2. **Attention pooling over per-residue tensors** — the most interesting lead *because* the fixed CDR3 mask failed: learned per-position weights would find whatever mix of CDR and framework actually matters, and are inspectable against the known CDR boundaries.
3. **Pool CDR1 and CDR2 too**. The CDR3-only failure implies the other loops carry signal; their flanking motifs are less conserved, so anchoring is harder.
4. **Cluster-aware calibration**, and whether within-class ρ differs between singleton clusters and large families — the latter may simply be easier.

6. Reproducibility

Commands in `README.md`. Seeds fixed (0/1/2, averaged, spread reported);

`torch.use_deterministic_algorithms(True, warn_only=True)`; versions in `uv.lock`; `embed_esm.py` writes `meta.json` (model id, layers, pooling rule, batch size, seed, device). `uv run pytest tests/ -q` — 11 tests covering pooling, the CDR3 anchors, undefined within-class ρ , and the sign convention.

Training and validation curves, project `sarasdragonz-university-of-toronto/her2-nanobody`: Task 1 `.../runs/mp8kfuxc`, Task 2 `.../runs/ih6bykb6`. Task 1 logs all three seeds into one run, so its `val_auroc` curve restarts three times rather than rising monotonically.

Cross-device check. Layer 24 was embedded twice independently, on a Colab T4 and on Apple MPS. Across all 19,367 sequences: max |difference| 1.8×10^{-4} , minimum cosine 0.99999982 — numerically interchangeable, and the downstream difference (0.9625 vs 0.9641) is inside seed noise. This matters because Colab had neither Transformer Engine, xformers nor flash-attn, so ESMC used pure-PyTorch LayerNorm, attention and RoPE; the vectors are not bit-identical to a fused-kernel run, and now that is measured rather than assumed.

One deviation from the brief. The suggested `AutoTokenizer / AutoModelForMaskedLM` path fails: `esm/_init_.py` exports nothing and never imports `esm.models.esmc`, so the custom classes are never registered with transformers and `tokenizer_class: EsmcTokenizer` cannot be resolved (`trust_remote_code=True` does not help — the class is in the installed package, not the model repo). `embed_esm.py` imports `EsmcTokenizer` and `EsmcForMaskedLM` directly.